

بسم الله الرحمن الرحيم

King Abdulaziz University
Faculty of Computing and information Technology
Computer Science Department



CPCS-214 Computer Architecture and Organization

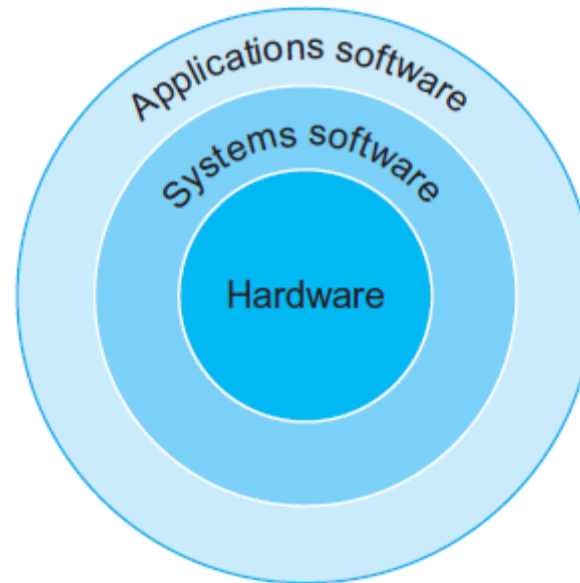
03 - Below Your Program

Introduction

- A typical application may
 - consist of millions of lines of codes
 - rely on sophisticated SW libraries that implement complex functions in support of the application
- HW can only execute extremely simple low-level instructions
- Interpret or translate high-level operations into simple computer instructions

Introduction

- Application SW: written in HLL
- System software
- Hardware: processor, memory, I/O controllers



Systems Software

- **Operating system**: interface between user's program and HW
 - Handling basic I/O operations
 - Allocating storage and memory
 - Protect sharing of computer among multiple applications using it simultaneously
- **Compilers**: translation of a program written in a HLL into instructions that HW can execute

From High-Level Language to Language of HW

- To speak to electronic HW, you need to send electrical signals
- Signals for computers to understand are *on and off*
 - *Computer* alphabet is just two letters **0** and **1**
 - *Binary digit or bit*
- Computers are slaves to our commands, which are called **instructions**

Instructions

- Collections of bits that computer understands and obeys
- Can be thought of as numbers

Instructions

- Collections of bits that computer understands and obeys
- Can be thought of as numbers
- For example, bits

1000110010100000

tell one computer to add two numbers

Assemblers

- First programmers communicated to computers in binary
 - Tedious
 - Quickly invented new notations closer to the way humans think
 - At first, these notations were translated to binary by hand
 - But this process was still tiresome

Assemblers

- First programmers communicated to computers in binary
 - Tedious
 - Quickly invented new notations closer to the way humans think
 - At first, these notations were translated to binary by hand
 - But this process was still tiresome
- Pioneers invented programs to translate from symbolic notation to binary
 - Assembler

Assembler

- Translate symbolic version of an instruction into binary version

Assembler

- Translate symbolic version of an instruction into binary version
- For example, programmer would write:

`add A, B`

- Assembler would translate this notation into:

Assembler

- Translate symbolic version of an instruction into binary version
- For example, programmer would write:

`add A, B`

- Assembler would translate this notation into:

`1000110010100000`

- This instruction tells computer to add two numbers A and B

Assembly Language

- Name coined for symbolic language, still used today
- In contrast, binary language that machine understands is **machine language**

Assembly Language

- Although a tremendous improvement, assembly language is still far from the notations a scientist might like to use
 - Requires programmer to write one line for every instruction that computer will follow
 - Forcing programmer to think like computer

High-level Programming Languages

- Recognition that a program could be written to translate a more powerful language into computer instructions was one of the great breakthroughs in the early days of computing

High-level Programming Languages

- Recognition that a program could be written to translate a more powerful language into computer instructions was one of the great breakthroughs in the early days of computing
- Programmers today owe their **productivity** and their sanity to creation of **high-level programming languages**
 - **Compiler**

Compiler

- Translate programs in high level languages into instructions

Compiler

- Translate programs in high level languages into instructions
- For example, compiler enables a programmer to write this high-level language expression:

$A + B$

- Compiler would compile it into assembly language instruction:

Compiler

- Translate programs in high level languages into instructions
- For example, compiler enables a programmer to write this high-level language expression:

$A + B$

- Compiler would compile it into assembly language instruction:

`add A, B`

Binary (Machine/Hardware) Language

```
Binary machine 000000001010001000000000100011000
language       000000001000001000010000000100001
program        10001101111000100000000000000000
(for MIPS)     100011100001001000000000000000100
               10101110000100100000000000000000
               101011011110001000000000000000100
               00000011111000000000000000001000
```

Assembly Language

Assembly
language
program
(for MIPS)

```
swap:
    multi $2, $5, 4
    add    $2, $4, $2
    lw     $15, 0($2)
    lw     $16, 4($2)
    sw     $16, 0($2)
    sw     $15, 4($2)
    jr     $31
```

Binary machine
language
program
(for MIPS)

```
000000001010001000000000100011000
00000000100000100001000000100001
10001101111000100000000000000000
100011100001001000000000000000100
10101110000100100000000000000000
101011011110001000000000000000100
00000011111000000000000000001000
```

High Level Language

High-level
language
program
(in C)

```
swap(int v[], int k)
{int temp;
  temp = v[k];
  v[k] = v[k+1];
  v[k+1] = temp;
}
```

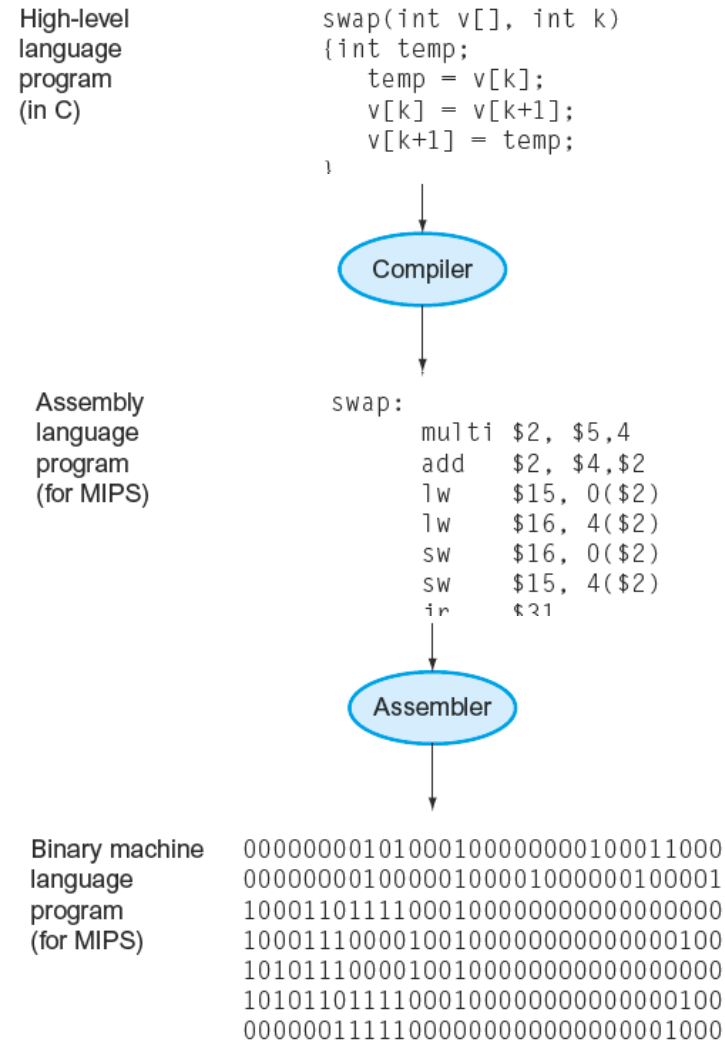
Assembly
language
program
(for MIPS)

```
swap:
    multi $2, $5, 4
    add   $2, $4, $2
    lw    $15, 0($2)
    lw    $16, 4($2)
    sw    $16, 0($2)
    sw    $15, 4($2)
    jr    $31
```

Binary machine
language
program
(for MIPS)

```
000000001010001000000000100011000
00000000100000100001000000100001
10001101111000100000000000000000
100011100001001000000000000000100
10101110000100100000000000000000
101011011110001000000000000000100
00000011111000000000000000001000
```

Program Compiled and Assembled



Benefits of HL-Programming Languages

- Programmer think in more natural language using English words and algebraic notation
- Improved programmer productivity
- Programs are independent of the computer on which they were developed